

PRINCIPI DELL'INGEGNERIA DEL SOFTWARE

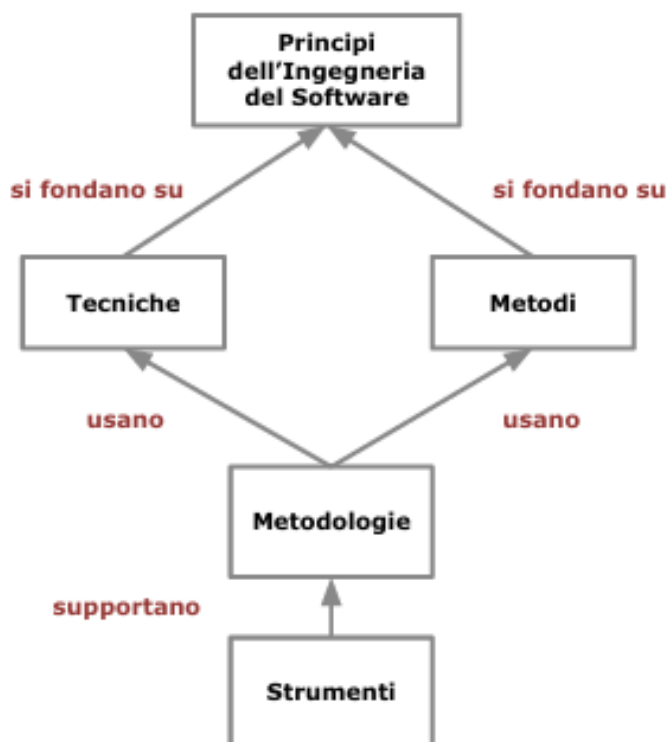
L'ingegneria del software si basa su principi fondamentali che servono per affrontare il problema principale dello sviluppo software, cioè la complessità.

Il software è un prodotto logico molto complesso, che deve essere progettato, sviluppato, controllato e modificato nel tempo.

La modularità è la pietra angolare della progettazione dei sistemi software. I principi descrivono proprietà desiderabili dei prodotti e dei processi in termini generali ed astratti e sono applicabili indipendentemente da:

- Contesto di sviluppo
- Tipo di sistema
- Ambiente di sviluppo
- Ambiente di utilizzo del software

Le metodologie possono essere supportate dagli strumenti che potenziano gli sviluppatori



RIGORE E FORMALITA'

L'Ingegneria del Software è un'attività creativa, ma deve essere praticata sistematicamente.

Il **RIGORE** rappresenta il complemento della creatività e consiste nella coerenza con le precondizioni, con gli obiettivi del prodotto da realizzare e con i metodi e le tecniche utilizzate.

Esistono diversi livelli di rigore:

Al livello più alto c'è la **FORMALITA'** ossia l'uso esclusivo di formalismi matematici per guidare lo sviluppo del software e per valutarlo

Es. Di prodotto

Precondizione

- ☐ $\exists$ un **Volo** con un **tempo di Durata** prevista
- ☐ Il volo è classificabile come: **Nazionale, Intracontinentale, Intercontinentale**

Postcondizione

- ☐ Definiti i Servizi di Ristoro da erogare

Rigoroso

- ☐ Se Durata del Volo < 1hh: distribuire bevande
- ☐ Se Durata del Volo > 1 hh e Tipo di Volo = Intracontinentale: Distribuire Bevande e Snack
- ☐ Se Durata del Volo > 1hh e Tipo di Volo = Intercontinentale: Distribuire Bevande e Pranzo
- ☐ Se Durata Volo > 8hh e Tipo di Volo = Intercontinentale: Distribuire Bevande, Pranzo e Colazione.

Consente di derivare rigorosamente e sistematicamente i dati per la verifica dinamica

Formale

☐ Condizioni

- Durata Volo : [1,8 [
- Tipo di Volo : [Nazionale; Intracontinentale; Intercontinentale]

☐ Azioni

- Distribuzione : [Bevande; Snack; Pranzo; Colazione]
- Distribuzione = F (Durata Volo, Tipo Volo)

Es. Di processo

La descrizione rigorosa delle attività e delle relazioni tra loro da eseguire durante lo sviluppo del software aiuta a gestire il prodotto e accertare i tempi ed i costi del processo

VANTAGGI

Il rigore consente di sviluppare software con buoni livelli di qualità, rendendo tracciabile e comprensibile la documentazione.

Il formalismo consente la gestione automatica dello sviluppo software con processo monitorato automaticamente ed anche eseguito in automatico.

SVANTAGGI

Difficile mantenere lo stesso livello di rigore spesso egli si comporta non conformemente.

SEPARAZIONE DEGLI INTERESSI

Consiste nel separare le decisioni relative ad aspetti diversi dello stesso problema. Questa separazione può avvenire in diversi modi:

- PER TEMPO : distingue le attività di analisi da quelle di progettazione
- PER CARATTERISTICA : prevede che si progetti prima la struttura del sistema e poi si migliori l'efficienza quando richiesto.
- PER PROSPETTIVE : Si struttura il sistema per assicurare una buona usabilità; si rivede poi una manutenzione economica implementando le specifiche riutilizzando software, si progetta l'interfaccia e infine si verifica quanto è efficiente.
- PER PARTI : Le applicazioni devono essere progettate con le seguenti parti:
 - ☐ Gestire l'interfaccia
 - ☐ Gestire le regole di dominio applicativo

- ❑ Gestire i dati persistenti

VANTAGGI E SVANTAGGI

Questo principio consente di affrontare problemi complessi analizzando un aspetto alla volta e riducendo il numero di decisioni simultanee.

Tuttavia, separare troppo gli aspetti può portare a perdere l'ottimizzazione complessiva, perché alcune decisioni sono tra loro correlate.

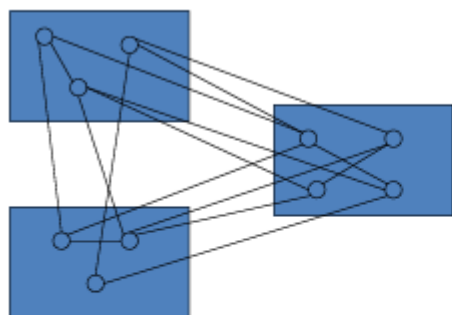
MODULARITA'

La modularità consiste nel suddividere il sistema software in moduli, ognuno dei quali realizza una parte delle funzioni del sistema e coopera con gli altri per raggiungere gli obiettivi complessivi.

La coesione misura quanto sono legati gli elementi interni di un modulo, mentre l'accoppiamento misura il livello di dipendenza tra moduli.

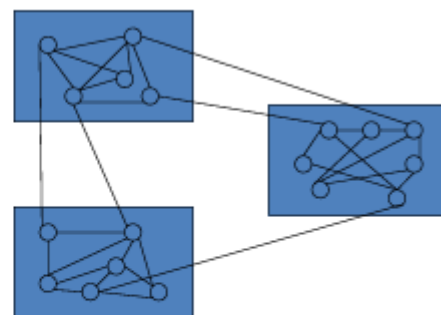
Un'applicazione si può costruire secondo due vie:

- Top Down : partendo dal sistema generale e suddividendolo in moduli
- Bottom up : partendo dai moduli per costruire il sistema completo.



Struttura ad
alto accoppiamento e
bassa coesione interna

Es.



Struttura con
alta coesione e
basso accoppiamento

VANTAGGI e SVANTAGGI

La modularità facilita la separazione degli interessi, la comprensione del sistema, la manutenzione e il riuso di componenti già esistenti. Tuttavia, una suddivisione eccessiva in moduli può rendere il sistema difficile da comprendere, soprattutto quando una funzione è distribuita tra molti moduli.

ANTICIPAZIONE DEL CAMBIAMENTO

Questo principio riguarda la capacità del software di adattarsi ai cambiamenti con poco sforzo, in poco tempo e con rischio limitato.

I cambiamenti possono derivare da una migliore comprensione del dominio applicativo, da cambiamenti nel dominio stesso o da cambiamenti tecnologici.

Questo principio è quello che differenzia considerevolmente lo sviluppo del software con la produzione di tutti gli altri tipi di beni

Per applicare questo principio è necessario progettare sistemi flessibili, prevedendo algoritmi alternativi o plugin che possano essere aggiunti senza modificare il sistema principale.

VANTAGGI e SVANTAGGI

Riduce i costi e i tempi di manutenzione e aumenta la riusabilità delle componenti. Tuttavia, richiede una progettazione molto accurata e una buona conoscenza del dominio applicativo, oltre all'uso di strumenti di gestione della configurazione software.

ASTRAZIONE

Consiste nell'estrarre gli aspetti più importanti di un problema trascurando i dettagli non rilevanti per lo scopo e può creare modelli che rappresentano la realtà in modo semplificato ma utile alla comprensione e alla progettazione

Le applicazioni software sono astrazioni di procedure manuali che automatizzano.

Es.

Esempio: In un programma che automatizza la fatturazione sono trascurati i particolari su come è organizzato l'ufficio fatturazione, se ha due o più persone, se è disposto su piani diversi dell'impresa, se è distribuito geograficamente

VANTAGGI e SVANTAGGI

L'astrazione consente anche di descrivere lo stesso sistema a diversi livelli di dettaglio e per diversi stakeholder, ad esempio utenti, analisti e progettisti. Tuttavia, un'astrazione progettata male può dare una visione distorta del sistema e far percepire una bassa qualità del software.

GENERALITA'

Il principio di generalità consiste nel cercare di trasformare un problema specifico in un problema più generale, la cui soluzione possa essere riutilizzata in altri contesti.

Generalizzare significa formulare un problema più generale che includa più casi particolari.

Qualche volta un problema più generale potrebbe essere più facile da risolvere di un suo caso speciale.

Es.

Stima dei consumi di uno strumento dinamico sia **$Stima = 4,5 * E ** (3,5 * ESP)$**

Le costanti sono state calcolate con dati derivati dall'uso di uno stesso motore per lavori analoghi

Questo problema si può riformulare come **$Stima = a * E ** (b * ESP) + k$** . Dove:

- ☐ (a, b, k) sono costanti che descrivono il contesto in cui è fatta la stima
- ☐ E, ESP sono le osservabili del sistema da stimare

VANTAGGI e SVANTAGGI

Questo permette di riusare moduli software in diversi contesti e di utilizzare prodotti già esistenti. Tuttavia, lo sviluppo di soluzioni generali comporta costi di sviluppo più elevati.

INCREMENTALITA'

L'incrementalità consiste nello sviluppare un sistema in modo graduale, attraverso incrementi successivi, ognuno dei quali aggiunge nuove funzionalità al sistema questo realizzato quindi per incremento.

Applicabile in: analisi, progettazione e integrazione

Es.

Si devono realizzare un sistema **S** con un set di funzioni $\{F_1, F_2, F_3, \dots F_n\}$

Il suo sviluppo si divide in tre incrementi

- $INCR_1 = \{F_1, F_2, F_3, \dots F_{k1}\};$
 $S_1 = INCR_1$
- $INCR_2 = \{F_{k1}, F_{k1+1}, F_{k1+2}, \dots F_{k2}\};$
 $S_2 = \{S_1 \cup INCR_2\}$
- $INCR_3 = \{F_{k2}, F_{k2+1}, F_{k2+2}, \dots F_n\};$
 $S = \{S_2 \cup INCR_3\}$

VANTAGGI e SVANTAGGI

Lo sviluppo incrementale consente di soddisfare rapidamente le esigenze più importanti degli utenti, ottenere feedback frequenti e ridurre il rischio di sviluppare un sistema non conforme ai requisiti. Tuttavia, aumenta il rischio di problemi durante l'integrazione tra gli incrementi e aumenta il costo dei test, che devono essere eseguiti dopo ogni incremento.

DAI PRINCIPI ALLO STREAM CODING

Si basa sull'idea che la specifica del sistema sia la “source of truth”, cioè la fonte principale da cui viene generato il codice.

Un approccio fallimentare è quello con documentazione frammentata, in cui l'AI formula ipotesi e si creano cicli infiniti di revisione del codice.

L'approccio corretto invece prevede documentazione chiara e completa, che consente un flusso deterministico di generazione del codice.

Introducendo la regola fondamentale: se si trova un bug, bisogna correggere la specifica e rigenerare il codice, evitando modifiche manuali che creano divergenza tra specifica e implementazione.

Lo Stream Coding applica i principi dell'ingegneria del software: rigore e formalità nella scrittura delle specifiche, separazione tra documentazione e implementazione, e anticipazione del cambiamento aggiornando la specifica e rigenerando i moduli.